



UNIVERSIDAD TECNOLÓGICA DE LEÓN
DESARROLLO PARA DISPOSITIVOS INTELIGENTES

IDGS903

CONSUMO DE UNA API REST DESDE ANDROID UTILIZANDO KOTLIN Y RETROFIT

Cruz De La Cruz Yaneth Guadalupe

Matricula: 23001496

Grupo: IDGS903

Lugar: León, Guanajuato

Fecha de desarrollo: 17/07/2026

Objetivo de la Práctica

Desarrollar un sistema de gestión bibliotecaria móvil multiplataforma mediante el uso de Kotlin y Jetpack Compose, enfocado en la centralización y administración automatizada de colecciones de libros. El propósito principal es construir una herramienta de software eficiente que simplifique la catalogación, actualización y consulta de ejemplares mediante una arquitectura modular, garantizando la consistencia de los datos enlazados a un servicio web externo.

Investigación

¿QUÉ ES UNA API REST?

Una API REST (Representational State Transfer) es un estilo de arquitectura de software utilizado para diseñar aplicaciones en red. Permite la comunicación entre un cliente (como tu app móvil) y un servidor mediante el protocolo HTTP. Se basa en recursos (datos) a los que se accede mediante URLs y se manipulan utilizando métodos estándar (GET, POST, PUT, DELETE).

¿QUÉ ES JSON?

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, fácil de leer y escribir para los humanos, y fácil de parsear y generar para las máquinas. Estructura la información en pares de clave-valor y arreglos, convirtiéndose en el estándar de facto para el envío de datos en las APIs REST modernas.

¿QUÉ ES RETROFIT?

Retrofit es un cliente HTTP tipo-seguro (type-safe) desarrollado por Square para Android y Java. Permite transformar una API REST externa en una interfaz de Kotlin mediante anotaciones, abstrayendo toda la complejidad de realizar conexiones de red, procesar las peticiones y manejar los hilos manualmente.

Investigación

¿QUÉ ES OKHTTP?

OkHttp es un cliente HTTP eficiente que también fue desarrollado por Square. Actúa de manera interna como la "capa de transporte" por defecto para Retrofit. Se encarga de la gestión de conexiones de red físicas, recuperación automática ante fallas de red, caché de respuestas y la interceptación de peticiones/respuestas útil para añadir tokens de seguridad o logs.

¿QUÉ ES GSON?

Gson es una librería desarrollada por Google que permite convertir objetos Java/Kotlin a su representación en formato JSON y viceversa (proceso conocido como serialización y deserialización). En Retrofit se integra como un Converter para mapear automáticamente las respuestas JSON del servidor a tus clases de datos (data classes).

¿QUÉ ES KTOR CLIENT? (INTRODUCCIÓN)

Ktor Client es un cliente HTTP asíncrono y multiplataforma desarrollado por JetBrains, diseñado específicamente para Kotlin. A diferencia de Retrofit (que nació en el ecosistema Java), Ktor está construido desde cero aprovechando al máximo las Coroutines y las características nativas de Kotlin, siendo muy popular en proyectos Kotlin Multiplatform (KMP).

¿QUÉ VENTAJAS OFRECE RETROFIT FRENTE A OTRAS LIBRERÍAS?

- **Abstracción declarativa:** Define las peticiones usando interfaces limpias con anotaciones simples.
- **Manejo automático de tipos:** Convierte automáticamente las respuestas JSON a tipos de datos nativos sin código manual intermedio.
- **Integración nativa:** Funciona de manera perfecta con OkHttp para el manejo de red y con Coroutines de Kotlin para la asincronía.
- **Ecosistema robusto:** Cuenta con una enorme comunidad, lo que asegura documentación masiva y soporte a largo plazo.

Investigación

¿CÓMO SE REALIZAN PETICIONES HTTP UTILIZANDO RETROFIT?

- **Definir la Data Class:** Crear la estructura de datos que recibirá el JSON.
- **Crear la Interfaz:** Definir las funciones que mapean los endpoints mediante anotaciones.
- **Configurar la Instancia:** Usar el Retrofit.Builder() especificando la URL base y el convertidor de JSON (como Gson).
- **Ejecutar la petición:** Invocar la función de la interfaz (usualmente dentro de un ViewModel usando Coroutines).

¿QUÉ SON LAS ANOTACIONES @GET, @POST, @PUT Y @DELETE?

Son anotaciones de Retrofit que corresponden directamente a los métodos del protocolo HTTP utilizados para interactuar con la API REST:

- **@GET:** Utilizado para consultar o recuperar información del servidor.
- **@POST:** Utilizado para enviar datos nuevos y crear un registro en el servidor.
- **@PUT:** Utilizado para actualizar de manera integral un registro existente en el servidor.
- **@DELETE:** Utilizado para eliminar un recurso específico en el servidor.

¿QUÉ ES UNA COROUTINE Y CÓMO SE UTILIZA CON RETROFIT?

Una Coroutine es un marco de diseño de concurrencia de Kotlin que permite ejecutar tareas asíncronas de forma simplificada y eficiente, sin bloquear el hilo principal (UI Thread). Con Retrofit, basta con añadir la palabra clave suspend antes de la función en la interfaz de la API. Al llamarla, la petición se ejecuta en un hilo secundario de fondo de manera transparente, permitiendo un código secuencial y fácil de leer.

Comparativa de librerías para consumo de APIs

Característica / Criterio	Retrofit	Ktor Client
Enfoque de diseño	Declarativo (Basado en Interfaces): Transforma una API REST en una interfaz de Kotlin mediante anotaciones sencillas.	Configurativo (Basado en bloques DSL): Configura dinámicamente las peticiones mediante bloques de código nativos de Kotlin.
Origen y Ecosistema	Desarrollado por Square. Nació en el entorno Java pero está totalmente adaptado y optimizado para Android.	Desarrollado por JetBrains (creadores de Kotlin). Diseñado desde cero de forma nativa para Kotlin.
Soporte Multiplataforma	Limitado. Está acoplado a la máquina virtual (JVM), por lo que se usa principalmente solo en Android/Java.	Completo. Diseñado específicamente para Kotlin Multiplatform (KMP), funcionando en Android, iOS, Desktop y Web.
Manejo de Asincronía	Soporta callbacks tradicionales y se integra de forma excelente con Coroutines usando la palabra clave suspend.	Utiliza Coroutines de manera obligatoria y nativa en su arquitectura interna para no bloquear los hilos.
Dependencias de Serialización	Requiere de conversores externos obligatorios (como Gson Converter) para procesar la información en formato JSON.	Utiliza extensiones configurables nativas de Kotlin (como Kotlinx Serialization) o conversores externos.
Curva de Aprendizaje	Muy baja. Al definir la estructura con anotaciones, es sumamente intuitivo y rápido de implementar en proyectos CRUD.	Media. Requiere que el desarrollador esté familiarizado con la sintaxis avanzada de bloques DSL y la configuración de plugins de Ktor.

Comparativa de librerías para consumo de APIs

¿POR QUÉ RETROFIT ES UNA DE LAS MÁS UTILIZADAS PARA ESTE TIPO DE APLICACIONES?

Basado en los requerimientos específicos de esta práctica de desarrollo nativo en Android Studio con arquitectura MVVM, Jetpack Compose y consumo de JSON, Retrofit destaca como la opción ideal por las siguientes razones clave:

1. **Reducción de Código Repetitivo (Boilerplate):** Al permitir definir las operaciones CRUD mediante anotaciones directas como `@GET`, `@POST`, `@PUT`, `@DELETE`, abstrae por completo la lógica de conexión. Esto permite que el desarrollador se enfoque en la lógica de negocio y en la interfaz de usuario en lugar de configurar manualmente las conexiones HTTP.
2. **Integración Nativa con la Arquitectura Recomendada:** Retrofit se acopla perfectamente con el ecosistema de componentes requeridos en la práctica. Trabaja de la mano con las Coroutines mediante funciones de suspensión para evitar congelar la pantalla, delega la serialización de forma transparente al Gson Converter, y entrega los datos limpios directamente al ViewModel.
3. **Estándar de la Industria y Estabilidad:** Es una librería madura con una inmensa comunidad detrás. Al ser promovida de manera directa en la documentación oficial de Android para el desarrollo de la capa de datos, garantiza estabilidad total, manejo eficiente de errores de red en conjunto con OkHttp, y una enorme facilidad para encontrar soluciones ante problemas de integración.

Arquitectura de la aplicación

MODELO DE BASE DE DATOS

```
class Libro(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    titulo = db.Column(db.String(150), nullable=False)
    autor = db.Column(db.String(100), nullable=False)
    isbn = db.Column(db.String(20), unique=True, nullable=False)
    ejemplares = db.Column(db.Integer, nullable=False, default=1)

    def to_dict(self):
        """Convierte el objeto en un diccionario para poder serializarlo a JSON"""
        return {
            "id": self.id,
            "titulo": self.titulo,
            "autor": self.autor,
            "isbn": self.isbn,
            "ejemplares": self.ejemplares
        }

# Crea las tablas en la base de datos si no existen
with app.app_context():
    db.create_all()
```

RUTAS DE LA API

• CONSULTAR TODOS LOS LIBROS (GET)

```
@app.route('/api/libros', methods=['GET'])
def obtener_libros():
    try:
        libros = Libro.query.all()
        return jsonify([libro.to_dict() for libro in libros]), 200
    except Exception as e:
        return jsonify({"error": "Error interno del servidor", "detalle": str(e)}), 500
```

Arquitectura de la aplicación

• AGREGAR UN LIBRO (POST)

```
@app.route('/api/libros', methods=['POST'])
def crear_libro():
    data = request.get_json()

    # Validaciones básicas del lado del servidor
    if not data or not all(k in data for k in ('titulo', 'autor', 'isbn', 'ejemplares')):
        return jsonify({"error": "Faltan campos obligatorios"}), 400

    # Validar que el ISBN no esté duplicado
    libro_existente = Libro.query.filter_by(isbn=data['isbn']).first()
    if libro_existente:
        return jsonify({"error": f"El ISBN '{data['isbn']}' ya se encuentra registrado"}), 400

    try:
        nuevo_libro = Libro(
            titulo=data['titulo'],
            autor=data['autor'],
            isbn=data['isbn'],
            ejemplares=int(data['ejemplares'])
        )
        db.session.add(nuevo_libro)
        db.session.commit()
        return jsonify(nuevo_libro.to_dict()), 201
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": "No se pudo guardar el libro", "detalle": str(e)}), 500
```

Arquitectura de la aplicación

• MODIFICAR UN LIBRO (PUT)

```
@app.route('/api/libros/<int:id>', methods=['PUT'])
def actualizar_libro(id):
    libro = Libro.query.get(id)
    if not libro:
        return jsonify({"error": "Libro no encontrado"}), 404
    data = request.get_json()
    try:
        if 'titulo' in data:
            libro.titulo = data['titulo']
        if 'autor' in data:
            libro.autor = data['autor']
        if 'isbn' in data:
            # Validar ISBN duplicado solo si se está modificando a uno diferente
            if data['isbn'] != libro.isbn:
                existente = Libro.query.filter_by(isbn=data['isbn']).first()
                if existente:
                    return jsonify({"error": "El nuevo ISBN ya está registrado por otro libro"}), 400
            libro.isbn = data['isbn']
        if 'ejemplares' in data:
            libro.ejemplares = int(data['ejemplares'])
        db.session.commit()
        return jsonify(libro.to_dict()), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": "No se pudo actualizar el libro", "detalle": str(e)}), 500
```

Arquitectura de la aplicación

• ELIMINAR UN LIBRO (DELETE)

```
@app.route('/api/libros/<int:id>', methods=['DELETE'])
def eliminar_libro(id):
    libro = Libro.query.get(id)
    if not libro:
        return jsonify({"error": "Libro no encontrado"}), 404
    try:
        db.session.delete(libro)
        db.session.commit()
        return jsonify({"mensaje": f"Libro '{libro.titulo}' eliminado correctamente"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": "No se pudo eliminar el libro", "detalle": str(e)}), 500
```

APLICACIÓN ANDROID

MODEL

```
package com.ejemplo.biblioteca.data.model
```

```
import com.google.gson.annotations.SerializedName
```

```
data class Libro(
    @SerializedName("id") val id: Int? = null,
    @SerializedName("titulo") val titulo: String? = "",
    @SerializedName("autor") val autor: String? = "",
    @SerializedName("isbn") val isbn: String? = "",
    @SerializedName("ejemplares") val ejemplares: Int? = 0,
    @SerializedName("genero") val genero: String? = "",
    @SerializedName("anioPublicacion") val anioPublicacion: Int? = 0
)
```

Arquitectura de la aplicación

LIBRO API SERVICE

```
package com.ejemplo.biblioteca.data.network
import com.ejemplo.biblioteca.data.model.Libro
import retrofit2.Response
import retrofit2.http.*

interface LibroApiService {
    @GET("libros")
    suspend fun obtenerLibros(): Response<List<Libro>>

    @POST("libros")
    suspend fun crearLibro(@Body libro: Libro): Response<Libro>

    @PUT("libros/{id}")
    suspend fun actualizarLibro(
        @Path("id") id: Int,
        @Body libro: Libro
    ): Response<Libro>

    @DELETE("libros/{id}")
    suspend fun eliminarLibro(@Path("id") id: Int): Response<Unit>
}
```

Arquitectura de la aplicación

RETROFIT

```
package com.ejemplo.biblioteca.data.network

import okhttp3.OkHttpClient
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

object RetrofitClient {

    private const val BASE_URL = "http://10.0.2.2:5000/api/"

    private val okHttpClient = OkHttpClient.Builder()
        .connectTimeout(5, TimeUnit.SECONDS)
        .readTimeout(5, TimeUnit.SECONDS)
        .build()

    val apiService: LibroApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .client(okHttpClient)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(LibroApiService::class.java)
    }
}
```

Arquitectura de la aplicación

PANTALLA DEL FORMULARIO

```
package com.ejemplo.biblioteca.ui.screens
```

```
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Book
import androidx.compose.material.icons.filled.CalendarMonth
import androidx.compose.material.icons.filled.Category
import androidx.compose.material.icons.filled.FormatListNumbered
import androidx.compose.material.icons.filled.Numbers
import androidx.compose.material.icons.filled.Person
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.ejemplo.biblioteca.data.model.Libro
import com.ejemplo.biblioteca.ui.viewmodel.LibroViewModel
import com.ejemplo.biblioteca.ui.viewmodel.UiState
import kotlinx.coroutines.launch
```

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PantallaFormulario(
    viewModel: LibroViewModel,
    libroId: Int?,
```

```
var titulo by remember { mutableStateOf("") }
var autor by remember { mutableStateOf("") }
var isbn by remember { mutableStateOf("") }
var ejemplares by remember { mutableStateOf("") }
var genero by remember { mutableStateOf("") }
var anioPublicacion by remember { mutableStateOf("") }

var tituloError by remember { mutableStateOf<String?>(null) }
var autorError by remember { mutableStateOf<String?>(null) }
var isbnError by remember { mutableStateOf<String?>(null) }
var ejemplaresError by remember { mutableStateOf<String?>(null) }
var generoError by remember { mutableStateOf<String?>(null) }
var anioPublicacionError by remember { mutableStateOf<String?>(null) }

val operacionState by viewModel.operacionState.collectAsState()
val snackbarHostState = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()
val scrollState = rememberScrollState()

LaunchedEffect(libroId) {
    if (libroId != null) {
        val state = viewModel.librosState.value
        if (state is UiState.Success) {
            state.data.find { it.id == libroId }?.let {
                titulo = it.titulo ?: ""
                autor = it.autor ?: ""
                isbn = it.isbn ?: ""
                ejemplares = it.ejemplares?.toString() ?: "0"
                genero = it.genero ?: ""
                anioPublicacion = it.anioPublicacion?.toString() ?: ""
            }
        }
    }
}

LaunchedEffect(operacionState) {
    if (operacionState is UiState.Error) {
        scope.launch {
            snackbarHostState.showSnackbar(
                message = (operacionState as UiState.Error).message,
                actionLabel = "Entendido"
            )
        }
        viewModel.resetOperacionState()
    }
}
```

```
    }  
  }  
}  
  
Scaffold(  
  topBar = {  
    TopAppBar(  
      title = { Text(if (libroId == null) "Registrar Libro" else "Modificar Ficha", fontWeight  
= FontWeight.Bold, color = Color.White) },  
      navigationIcon = {  
        IconButton(onClick = onNavigateBack) {  
          Icon(Icons.Default.ArrowBack, contentDescription = "Regresar", tint =  
Color.White)  
        }  
      },  
      colors = TopAppBarDefaults.topAppBarColors(  
        containerColor = MaterialTheme.colorScheme.primary  
      )  
    )  
  },  
  snackbarHost = { SnackbarHost(hostState = snackbarHostState) }  
) { paddingValues ->  
  Column(  
    modifier = Modifier  
      .fillMaxSize()  
      .background(MaterialTheme.colorScheme.background)  
      .padding(paddingValues)  
      .padding(16.dp)  
      .verticalScroll(scrollState),  
    verticalArrangement = Arrangement.spacedBy(12.dp)  
  ) {  
    Text(  
      text = "Detalles de la Obra",  
      style = MaterialTheme.typography.titleMedium,  
      color = MaterialTheme.colorScheme.primary,  
      fontWeight = FontWeight.Bold  
    )  
  }  
}
```

// TÍTULO

```
OutlinedTextField(  
  value = titulo,  
  onChange = { titulo = it; tituloError = null },  
  label = { Text("Título de la obra") },  
  leadingIcon = { Icon(Icons.Default.Book, contentDescription = null, tint =  
MaterialTheme.colorScheme.primary) },  
  isError = tituloError != null,  
  supportingText = { tituloError?.let { Text(it) } },  
  shape = RoundedCornerShape(12.dp),  
  modifier = Modifier.fillMaxWidth()  
)
```

// AUTOR

```
OutlinedTextField(  
  value = autor,  
  onChange = { autor = it; autorError = null },  
  label = { Text("Autor / Escritor") },  
  leadingIcon = { Icon(Icons.Default.Person, contentDescription = null, tint =  
MaterialTheme.colorScheme.primary) },  
  isError = autorError != null,  
  supportingText = { autorError?.let { Text(it) } },  
  shape = RoundedCornerShape(12.dp),  
  modifier = Modifier.fillMaxWidth()  
)
```

// GÉNERO

```
OutlinedTextField(  
  value = genero,  
  onChange = { genero = it; generoError = null },  
  label = { Text("Género Literario") },  
  leadingIcon = { Icon(Icons.Default.Category, contentDescription = null, tint =  
MaterialTheme.colorScheme.primary) },  
  isError = generoError != null,  
  supportingText = { generoError?.let { Text(it) } },  
  shape = RoundedCornerShape(12.dp),  
  modifier = Modifier.fillMaxWidth()  
)
```

// AÑO DE PUBLICACIÓN

```
OutlinedTextField(  
  value = anioPublicacion,  
  onChange = { input ->  
    if (input.all { it.isDigit() }) {  
      anioPublicacion = input  
      anioPublicacionError = null  
    }  
  },  
  label = { Text("Año de Publicación") },  
  leadingIcon = { Icon(Icons.Default.CalendarMonth, contentDescription = null, tint =  
MaterialTheme.colorScheme.primary) },  
  keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),  
  isError = anioPublicacionError != null,  
  supportingText = { anioPublicacionError?.let { Text(it) } },  
  shape = RoundedCornerShape(12.dp),  
  modifier = Modifier.fillMaxWidth()  
)
```

// ISBN

```
OutlinedTextField(  
  value = isbn,  
  onChange = { isbn = it; isbnError = null },  
  label = { Text("ISBN (Código)") },  
  leadingIcon = { Icon(Icons.Default.Numbers, contentDescription = null, tint =  
MaterialTheme.colorScheme.primary) },  
  isError = isbnError != null,  
  supportingText = { isbnError?.let { Text(it) } },  
  shape = RoundedCornerShape(12.dp),  
  modifier = Modifier.fillMaxWidth()  
)
```

Arquitectura de la aplicación

PANTALLA PRINCIPAL

```
package com.ejemplo.biblioteca.ui.screens
```

```
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.ejemplo.biblioteca.data.model.Libro
import com.ejemplo.biblioteca.ui.viewmodel.LibroViewModel
import com.ejemplo.biblioteca.ui.viewmodel.UiState
```

```
@OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
```

```
fun PantallaPrincipal(
    viewModel: LibroViewModel,
    onNavigateToFormulario: (id: Int?) -> Unit
) {
    val librosState by viewModel.librosState.collectAsState()
    var libroAEliminar by remember { mutableStateOf<Libro?>(null) }

    Scaffold(
        topBar = {
            CenterAlignedTopAppBar(
                title = {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(
                            imageVector = Icons.Default.MenuBook,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.secondary,
                            modifier = Modifier.padding(end = 8.dp)
                        )
                        Text(
                            "Mi Biblioteca",
                            style = MaterialTheme.typography.titleMedium,
                            color = Color.White
                        )
                    }
                },
                colors = TopAppBarDefaults.centerAlignedTopAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primary
                )
            )
        },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { onNavigateToFormulario(null) },
                containerColor = MaterialTheme.colorScheme.primary,
                contentColor = MaterialTheme.colorScheme.secondary,
                shape = CircleShape
            ) {

```

```
) {  
    Icon(Icons.Default.Add, contentDescription = "Agregar Libro", modifier =  
Modifier.size(28.dp))  
}  
}  
{ paddingValues ->  
    Box(  
        modifier = Modifier  
            .fillMaxSize()  
            .background(MaterialTheme.colorScheme.background)  
            .padding(paddingValues),  
        contentAlignment = Alignment.Center  
    ) {  
        when (val state = librosState) {  
            is UiState.Loading -> CircularProgressIndicator(color =  
MaterialTheme.colorScheme.primary)  
            is UiState.Success -> {  
                if (state.data.isEmpty()) {  
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {  
                        Icon(  
                            Icons.Default.MenuBook,  
                            contentDescription = null,  
                            modifier = Modifier.size(72.dp),  
                            tint = MaterialTheme.colorScheme.primary.copy(alpha = 0.4f)  
                        )  
                        Spacer(modifier = Modifier.height(12.dp))  
                        Text("Los estantes están vacíos.", fontSize = 16.sp, color =  
MaterialTheme.colorScheme.onSurface.copy(alpha = 0.7f))  
                    }  
                } else {  
                    LazyColumn(  
                        modifier = Modifier.fillMaxSize().padding(14.dp),  
                        verticalArrangement = Arrangement.spacedBy(16.dp)  
                    ) {  
                        items(state.data) { libro ->  
                            CardLibro(  
                                libro = libro,  
                                onEdit = { onNavigateToFormulario(libro.id) },  
                                onDelete = { libroAEliminar = libro }  
                            )  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```

libroAEliminar?.let { libro ->
    AlertDialog(
        onDismissRequest = { libroAEliminar = null },
        title = { Text("Archivar / Eliminar") },
        text = { Text("¿Deseas retirar permanentemente '${libro.titulo ? : "este libro"}' del inventario?" ) },
        confirmButton = {
            TextButton(onClick = {
                libro.id?.let { viewModel.eliminarLibro(it) }
                libroAEliminar = null
            }) {
                Text("Eliminar", color = MaterialTheme.colorScheme.error, fontWeight =
FontWeight.Bold)
            }
        },
        dismissButton = {
            TextButton(onClick = { libroAEliminar = null }) {
                Text("Cancelar")
            }
        }
    )
}
}

```

@Composable

```

fun CardLibro(libro: Libro, onEdit: () -> Unit, onDelete: () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(22.dp),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
        border = androidx.compose.foundation.BorderStroke(
            1.dp,
            MaterialTheme.colorScheme.secondary.copy(alpha = 0.25f)
        )
    ) {

```

```
Row(modifier = Modifier.fillMaxWidth()) {  
    // BARRA DE ACENTO LATERAL  
    Box(  
        modifier = Modifier  
            .width(6.dp)  
            .fillMaxHeight()  
            .background(MaterialTheme.colorScheme.primary)  
    )  
  
    Row(  
        modifier = Modifier.padding(20.dp).fillMaxWidth(),  
        horizontalArrangement = Arrangement.SpaceBetween,  
        verticalAlignment = Alignment.Top  
    ) {  
        Column(modifier = Modifier.weight(1f)) {  
  
            // TÍTULO DEL LIBRO  
            Text(  
                text = libro.titulo ?: "Sin título",  
                style = MaterialTheme.typography.titleLarge,  
                color = MaterialTheme.colorScheme.primary  
            )  
  
            Spacer(modifier = Modifier.height(6.dp))  
  
            // AUTOR CON AVATAR CIRCULAR (INICIAL)  
            Row(verticalAlignment = Alignment.CenterVertically) {  
                Box(  
                    modifier = Modifier  
                        .size(26.dp)  
                        .clip(CircleShape)  
                        .background(MaterialTheme.colorScheme.secondary.copy(alpha = 0.35f)),  
                    contentAlignment = Alignment.Center  
                ) {  
                    Text(  
                        text = (libro.autor?.trim())?.firstOrNull()?.uppercase() ?: "?",  
                        fontSize = 12.sp,  
                        fontWeight = FontWeight.Bold,  
                        color = MaterialTheme.colorScheme.primary  
                    )  
                }  
            }  
        }  
    }  
}
```

```
// DETALLES ADICIONALES (AÑO, ISBN)
```

```
Text(
```

```
    text = "AÑO: ${libro.anioPublicacion ?: "N/A"} • ISBN: ${libro.isbn ?: "N/A"}",
```

```
    style = MaterialTheme.typography.bodySmall,
```

```
    color = Color.Gray
```

```
)
```

```
Spacer(modifier = Modifier.height(10.dp))
```

```
// BADGE DE DISPONIBILIDAD (PILL)
```

```
val totalEjemplares = libro.ejemplares ?: 0
```

```
val esBajo = totalEjemplares <= 2
```

```
val colorStock = if (esBajo) Color(0xFFC0392B) else Color(0xFF2E9E5B)
```

```
    val textoStock = if (totalEjemplares == 1) "¡Último ejemplar!" else "$totalEjemplares unidades"
```

```
Row(
```

```
    verticalAlignment = Alignment.CenterVertically,
```

```
    modifier = Modifier
```

```
        .clip(RoundedCornerShape(50))
```

```
        .background(colorStock.copy(alpha = 0.12f))
```

```
        .padding(horizontal = 10.dp, vertical = 5.dp)
```

```
) {
```

```
    Icon(
```

```
        imageView = if (esBajo) Icons.Default.Warning else Icons.Default.CheckCircle,
```

```
        contentDescription = null,
```

```
        tint = colorStock,
```

```
        modifier = Modifier.size(14.dp)
```

```
)
```

```
Spacer(modifier = Modifier.width(5.dp))
```

```
Text(
```

```
    text = textoStock,
```

```
    fontSize = 12.sp,
```

```
    fontWeight = FontWeight.Bold,
```

```
    color = colorStock
```

```
)
```

```
}
```

```
}
```

Arquitectura de la aplicación

MAIN ACTIVITY

```
package com.ejemplo.biblioteca
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.lightColorScheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.ui.graphics.Color
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.ejemplo.biblioteca.ui.screens.PantallaFormulario
import com.ejemplo.biblioteca.ui.screens.PantallaPrincipal
import com.ejemplo.biblioteca.ui.viewmodel.LibroViewModel
```

```
// === PALETA DE COLORES "BIBLIOTECA ROSA" ===
```

```
val WarmBrown = Color(0xFFC2607F)    // Rosa Empolvado (Color Principal)
val DeepWood = Color(0xFF5C2A3A)     // Rosa Vino Oscuro (textos/contraste)
val WarmBeige = Color(0xFFFFCEE2)     // Rosa Pastel Suave (Fondo)
val AccentGold = Color(0xFFE8A0B4)    // Rosa Coral (Detalles/Acentos)
val SoftSage = Color(0xFFF6DCE2)     // Rosa Claro (Contenedores secundarios)
```

```
private val BibliotecaLightColorScheme = lightColorScheme(
    primary = WarmBrown,
    onPrimary = Color.White,
    secondary = AccentGold,
    background = WarmBeige,
    surface = Color.White,
```

```
private val BibliotecaLightColorScheme = lightColorScheme(
    primary = WarmBrown,
    onPrimary = Color.White,
    secondary = AccentGold,
    background = WarmBeige,
    surface = Color.White,
    surfaceVariant = SoftSage,
    onSurface = DeepWood
)

class MainActivity : ComponentActivity() {
    private val viewModel: LibroViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme(
                colorScheme = BibliotecaLightColorScheme
            ) {
                Surface(color = MaterialTheme.colorScheme.background) {
                    val navController = rememberNavController()

                    NavHost(navController = navController, startDestination = "principal") {
                        composable("principal") {
                            PantallaPrincipal(
                                viewModel = viewModel,
                                onNavigateToFormulario = { id ->
                                    if (id != null) {
                                        navController.navigate("formulario?id=$id")
                                    } else {
                                        navController.navigate("formulario")
                                    }
                                }
                            )
                        }
                    }
                }
            }
        }
    }
}
```

Explicación del funcionamiento

El funcionamiento de este sistema es bastante sencillo e intuitivo desde el punto de vista del usuario, pero por detrás realiza un flujo muy completo de comunicación. En cuanto abrimos la aplicación por primera vez, la pantalla principal se conecta de inmediato con el servidor web para solicitar la lista de libros actualizada. Mientras el teléfono espera la respuesta de la base de datos, el usuario ve una animación de carga en pantalla que indica que el sistema está trabajando. Una vez que los datos llegan, esa pantalla de carga desaparece para mostrar la lista completa de libros acomodada en tarjetas individuales. Cada tarjeta muestra de forma clara el título, el autor, el género literario destacado en una pequeña etiqueta, el año de publicación y un aviso de disponibilidad en color verde o rojo que advierte cuántos ejemplares quedan en el inventario.

Para agregar o editar libros aprovechamos una sola pantalla, lo que hace que todo el proceso sea muy fluido. Si decidimos registrar un libro nuevo presionando el botón flotante con el icono de suma, el formulario se abre completamente limpio para empezar a escribir. Pero si lo que queremos es modificar una obra y tocamos el icono de edición, la aplicación viaja a ese mismo formulario llevando consigo el identificador del libro para buscar sus datos en el sistema y rellenar cada campo de texto automáticamente. Al presionar el botón de guardar, la aplicación realiza un filtro de seguridad rápido para comprobar que no falte ningún dato obligatorio y que tanto el año como la cantidad de copias sean números lógicos. Si todo es correcto, se envía la información para actualizar la base de datos y la aplicación nos regresa de inmediato a la lista principal con los cambios ya aplicados.

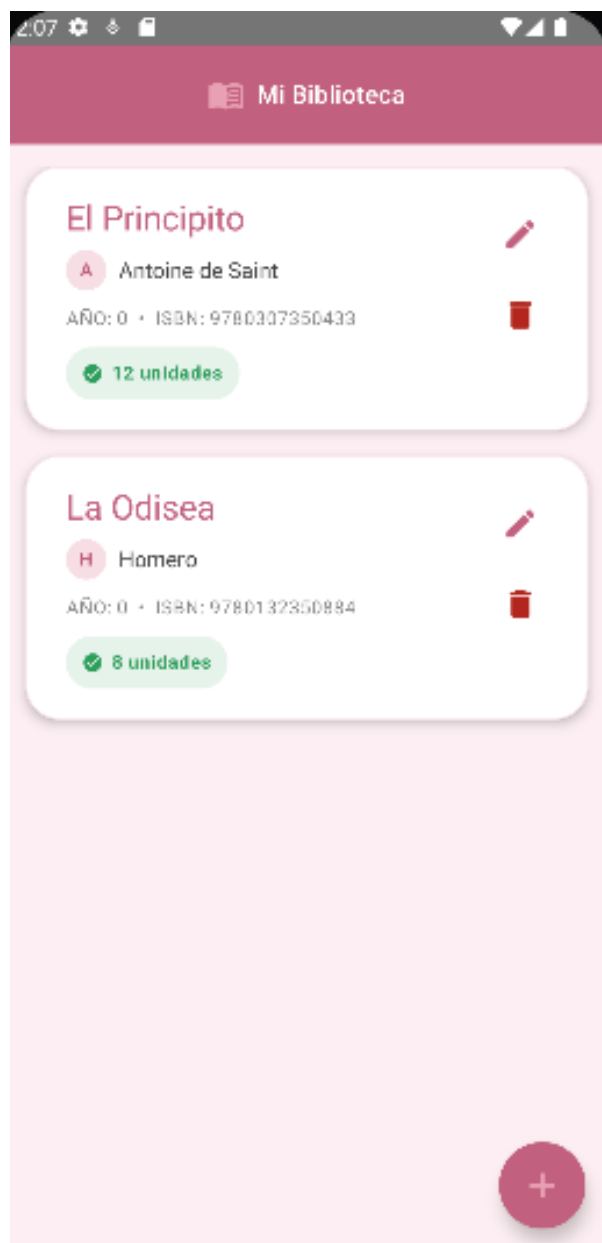
Finalmente, el sistema cuenta con medidas de seguridad para evitar errores humanos o fallas técnicas. Si intentamos borrar un libro, la aplicación no lo elimina de golpe; primero nos muestra un cuadro de confirmación para asegurarnos de que no lo estemos borrando por accidente. Además, si en cualquier momento nos quedamos sin internet o el servidor llega a fallar, la interfaz de la aplicación nunca se congela ni se cierra bruscamente; en su lugar, se activa un discreto aviso flotante en la parte inferior de la pantalla que advierte sobre el problema de conexión, permitiendo que podamos corregir los datos o reintentar la acción cuando la red regrese a la normalidad.

Resultados

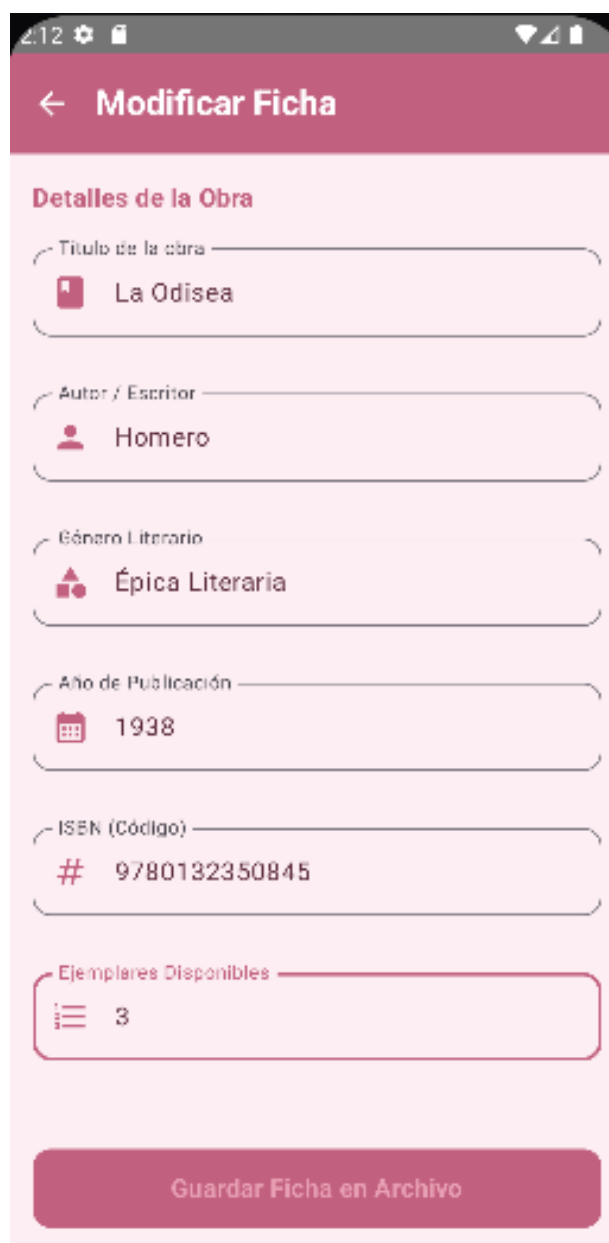
DATOS EN JSON

operaciones CRUD

LISTA DE REGISTROS



MODIFICAR REGISTROS

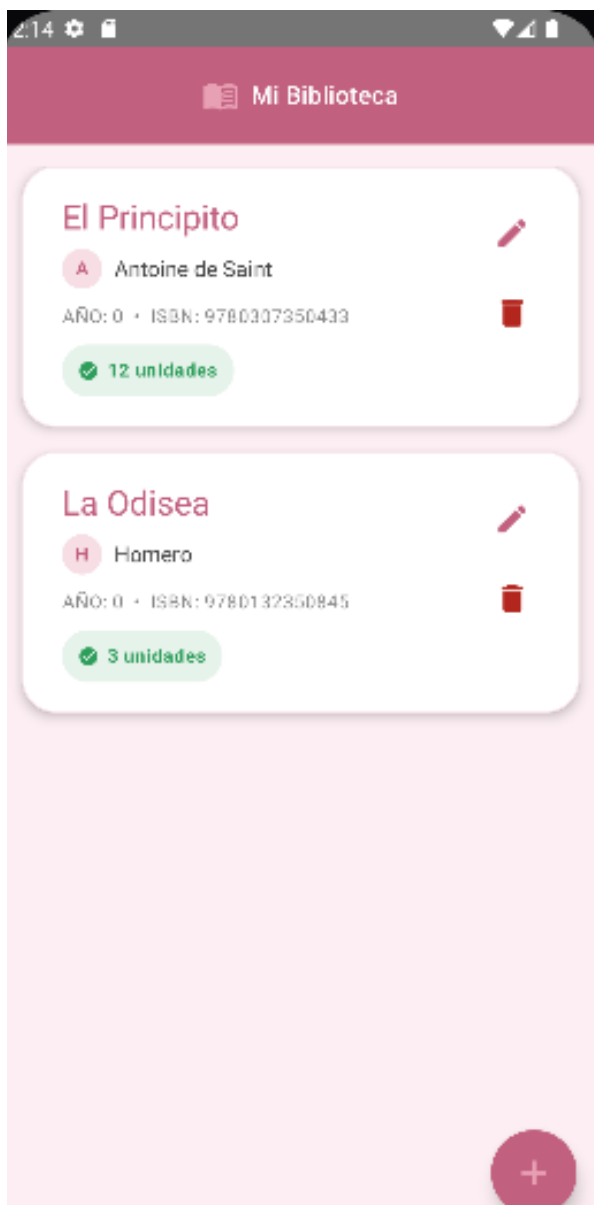


The screenshot shows a mobile application interface titled 'Modificar Ficha'. It contains a form to edit book details. The form fields are: 'Título de la obra' (La Odisea), 'Autor / Escritor' (Homero), 'Género Literario' (Épica Literaria), 'Año de Publicación' (1938), 'ISBN (Código)' (9780132350845), and 'Ejemplares Disponibles' (3). A red button at the bottom is labeled 'Guardar Ficha en Archivo'.

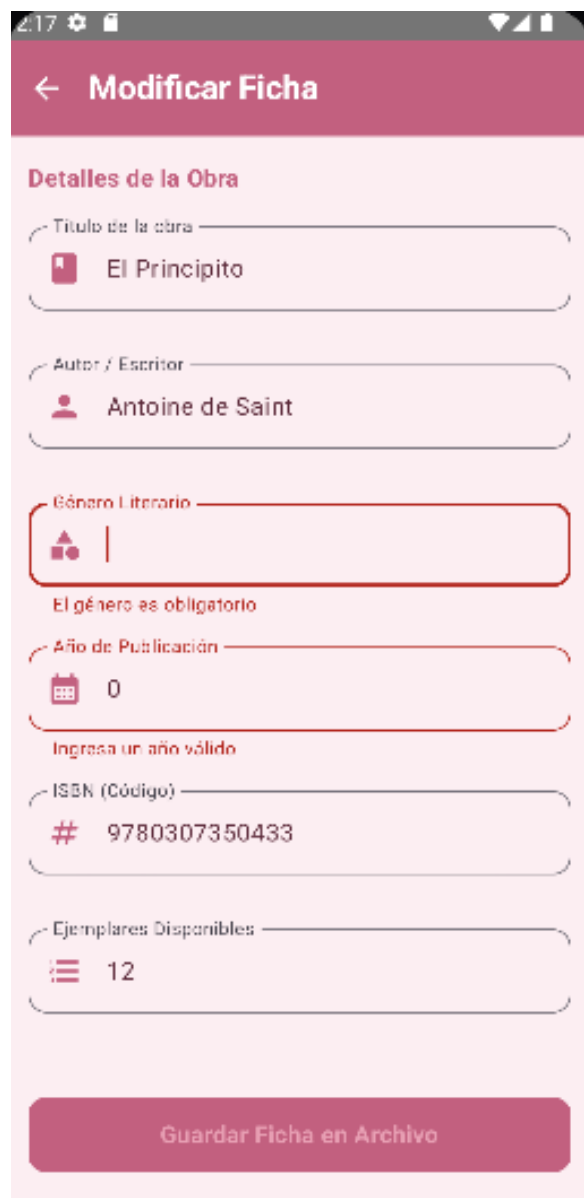
Field	Value
Título de la obra	La Odisea
Autor / Escritor	Homero
Género Literario	Épica Literaria
Año de Publicación	1938
ISBN (Código)	9780132350845
Ejemplares Disponibles	3

operaciones CRUD

MODIFICACIÓN CORRECTA



VALIDACIONES



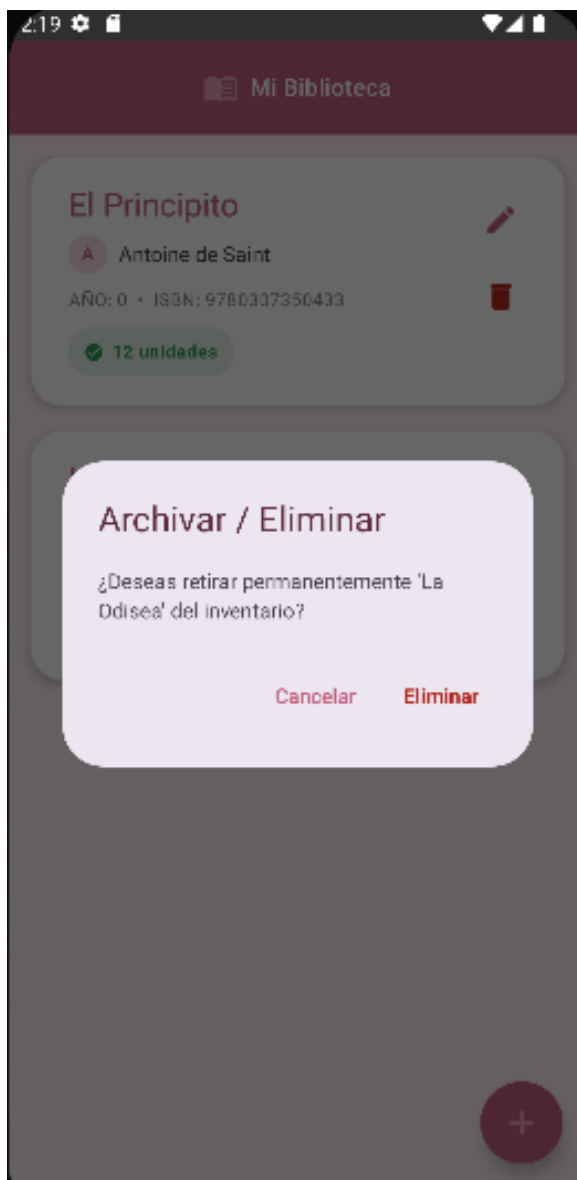
The screenshot shows the 'Modificar Ficha' app interface. It has a header with a back arrow and the text 'Modificar Ficha'. Below this, there's a section titled 'Detalles de la Obra'. The form contains several fields with validation errors:

- Título de la obra:** El Principito (Valid)
- Autor / Escritor:** Antoine de Saint (Valid)
- Género Literario:** (Empty) - Error: El género es obligatorio
- Año de Publicación:** 0 - Error: Ingresa un año válido
- ISBN (Código):** # 9780307350433 (Valid)
- Ejemplares Disponibles:** 12 (Valid)

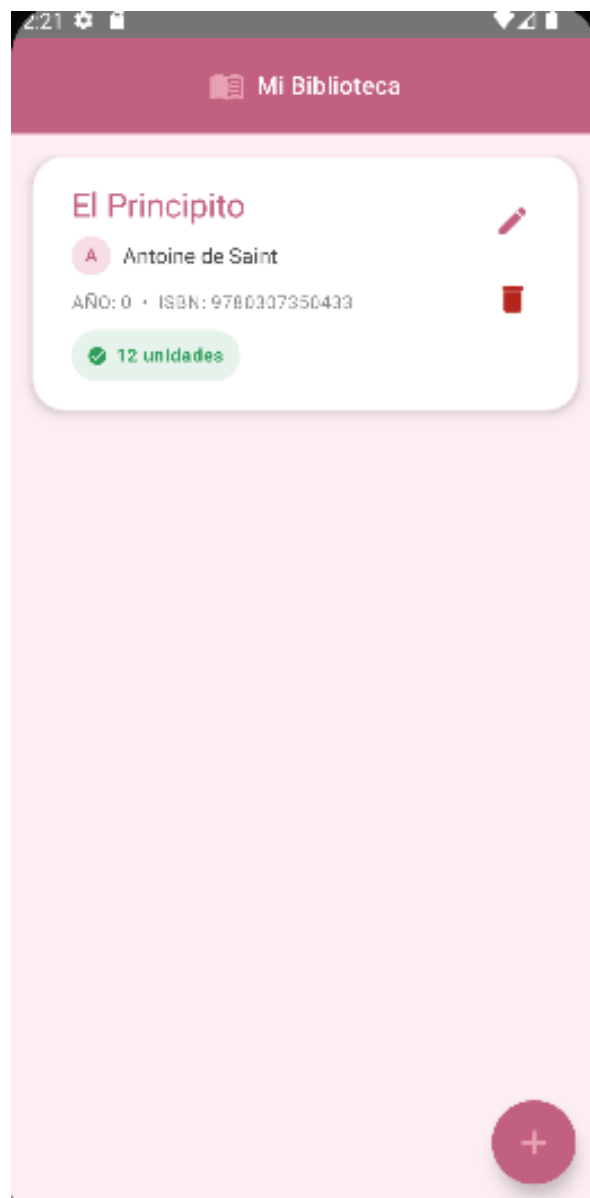
At the bottom, there is a button labeled 'Guardar Ficha en Archivo'.

operaciones CRUD

ELIMINAR REGISTRO



ELIMINACIÓN CORRECTA



Problemas encontrados y su solución

EXCEPCIONES POR VALORES NULOS EN EL MODELO DE DATOS

Problema:

Al interactuar con registros antiguos en la base de datos a través de la API, algunos campos nuevos como genero o añoPublicacion venían como nulos (null) en la respuesta JSON de los servicios web, lo que provocaba que la aplicación se cerrara inmediatamente al intentar renderizar la lista.

Solución: Se implementó un blindaje de nulos en el modelo de datos de Kotlin (Libro.kt) utilizando valores predeterminados seguros (= "" o = 0) y aplicando el operador Elvis (?:) en los componentes gráficos de Compose. Esto aseguró que la interfaz de usuario tuviera siempre un valor de cadena válido para pintar, evitando colapsos inesperados en la app.

BLOQUEO DE LA INTERFAZ POR OPERACIONES DE RED SÍNCRONAS

Problema: Al intentar registrar, modificar o eliminar un libro, la interfaz de la aplicación sufría de congelamientos momentáneos (ANR - Application Not Responding) mientras esperaba la respuesta de confirmación del servidor.

Solución: Se estructuró el flujo de operaciones asíncronas en el LibroViewModel utilizando Corrutinas de Kotlin y flujos reactivos (StateFlow). Al transferir la carga de trabajo de las llamadas de red al despachador de hilos de fondo (Dispatchers.IO), la interfaz de usuario se mantuvo interactiva y responsiva en todo momento, mostrando indicadores de carga únicamente cuando era necesario.

RETROALIMENTACIÓN SILENCIOSA ANTE FALLAS DE CONEXIÓN

Problema: Si el backend se caía o el dispositivo perdía el acceso a internet durante una actualización, el formulario fallaba en silencio y el usuario no sabía si sus cambios se habían guardado o no.

Solución: Se incorporó un sistema de control de errores mediante una clase sellada UiState que encapsula los estados de Loading, Success y Error. Al interceptar las fallas del servidor en el ViewModel, estas se canalizaron a la pantalla a través de un componente SnackbarHost, notificando de forma visual y clara al usuario sobre el estado de su solicitud.

Conclusiones

A través de la realización de esta aplicación de biblioteca, adquirí una comprensión profunda sobre la integración de aplicaciones móviles con APIs externas y la importancia de construir interfaces tolerantes a fallos. Uno de los mayores aprendizajes fue descubrir cómo Jetpack Compose maneja de manera reactiva la propagación de estados, lo que me obligó a diseñar flujos de datos mucho más limpios y estructurados. Resolver los desafíos relacionados con la conversión de tipos y la protección contra valores nulos me demostró que el desarrollo móvil requiere una planificación minuciosa de la arquitectura del software. Esta experiencia ha fortalecido significativamente mis habilidades para crear herramientas de software funcionales, adaptables y con altos estándares de experiencia de usuario.